

Practica 7: Sistemas y Tipos de Datos

1) Sistemas de tipos

1. Un sistema de tipos es un conjunto de reglas usadas por un lenguaje para estructurar y organizar sus tipos.

Su objetivo es escribir programas seguros.

2. Basado en la Conversión Implícita de Tipos:

- Tipado Fuerte: el lenguaje no realiza conversiones automáticas e implícitas entre tipos incompatibles. Si quieres operar un numero con un texto, debes castearlo. Si no lo haces, el sistema lanza un error.
- Tipado Débil: el lenguaje realiza conversiones implícitas automáticamente (coercion) para intentar que la operación funcione incluso si los tipos son conceptualmente distintos

Basado en la Seguridad de Memoria:

- Tipado Fuerte: un lenguaje es fuertemente tipado si garantiza que una variable solo puede contener valores del tipo que se le asignó, impidiendo que se corrompa la memoria o se interpreten bytes de forma errónea.
- Tipado Débil: permite "engañar" al sistema para tratar un bloque de memoria asignado a un tipo como si fuera de otro distinto (por ejemplo, leer un puntero de memoria como si fuera un entero), lo que puede causar fallos catastróficos o problemas de seguridad.

3. Tipado Estático: la comprobación de tipos se realiza durante la fase de compilación, es decir, antes de que el programa se ejecute. Si encuentra una inconsistencia, el programa no compila y genera un error. Ejemplos:

- 1) Java: obligado a declarar explícitamente el tipo de cada variable antes de usarla.
`int numero = 10; numero = "Hola";`
Si compilo el código, el compilador detendrá el proceso y arrojará un error de tipos antes de que el programa pueda iniciar.
- 2) TypeScript: el compilador de TypeScript examina el código en busca de errores de tipo. Si definís que una función recibe un número y le pasas un texto, el editor y el compilador te avisarán inmediatamente del error en tiempo de desarrollo, impidiendo que el código defectuoso llegue a producción.

Tipado Dinámico: la comprobación de tipos se realiza en tiempo de ejecución (*runtime*), es decir, mientras el programa se está ejecutando. Las variables no están amarradas a un tipo de dato específico. Ofrece una velocidad de desarrollo mucho mayor y requiere escribir menos código estructurado. Ejemplos:

- 1) JavaScript: no necesitas ni puedes declarar si una variable es un entero o un texto. El motor descubre el tipo sobre la marcha. Como la variable no está atada al tipo, reasignarla a algo diferente es totalmente válido. Ej:

et x = 42; // En este momento, x almacena un Number x = "Ahora texto";

- 2) Python: combina el tipado dinámico con el tipado fuerte. Al igual que JavaScript.
Ej: variable = [1, 2, 3]; variable = 8.5

2) Tipos de datos

1. El tipo de dato es un conjunto de valores y de operaciones que se pueden utilizar para manipularlos.
2. Un tipo predefinido elemental, también conocido como tipos primitivos, son aquellos que ya vienen integrados de forma estándar en el lenguaje de programación. Son los bloques de construcción más básicos. Reflejan el comportamiento del hardware y son una abstracción de él. Como los enteros, reales, caracteres, booleanos, string.
3. Un tipo definido por el usuario son tipos de datos complejos que el programador crea combinando los tipos predefinidos (primitivos) u otros tipos ya existentes. Nacen de la necesidad de modelar conceptos del mundo real o estructuras más abstractas que un simple número o texto no pueden representar por sí solos. Como el tipo Tiempo.

3) Tipos compuestos:

1.
 - Producto Cartesiano: utiliza el concepto matemático para crear estructuras o registros.
 - Correspondencia Finita: también conocida como mapeo, asocia de forma única cada elemento del dominio con un elemento del rango. Se refiere a los arreglos o listas.
 - Union / Union discriminada: espacio de memoria compartido que guarda el dato A O dato B. La union discriminada es la versión segura que añade una etiqueta para saber que dato está en memoria
 - Recursión: un tipo de dato recursivo T se define como una estructura que puede contener componentes del tipo T. Permite que las estructuras se conecten a sí mismas para crecer infinitamente en ejecución (listas, árboles, grafos).
2.
 1. Java : tipo de dato definido por el usuario (clase Persona) compuesto (formado por varios datos más pequeños) basado en producto cartesiano (conjunto de textos, apellido y nombre, y números, edad)
 2. C: Tipos de datos compuestos definidos por el usuario que implementan un tipo recursivo, porque `_nodoLista` contiene dentro de su definición `struct _nodoLista` *siguiente para poder encadenarse a sí misma. Producto cartesiano porque

nodoLista y Lista son structs; y tipo de dato abstracto, aunque conceptualmente el void*dato no cambia la naturaleza algebraica de la estructura, en la practica el uso de punteros genéricos emula un comportamiento polimórfico.

3. C: Es un tipo de dato compuesto basado en Uniones (*Sum Type*). Es un tipo definido por el usuario donde los campos no coexisten. La variable puede contener un número entero O un carácter, pero solo uno de ellos a la vez, compartiendo el mismo espacio de memoria.
4. Ruby: pertenece a la clase de Correspondencia Finita. Es una estructura de datos basada en pares clave-valor (llamada Hash en Ruby). Cada clase única se mapea o corresponde con un único valor específico.
5. PHP: pertenece a la clase de Tipos de Función. Una función se considera un tipo de dato por si misma. En este caso representa un mapeo o transformación que toma un valor de un tipo de entrada (dominio) y devuelve un valor de un tipo de salida (codominio). $X \rightarrow Y$
6. Python: pertenece a la clase de Producto Cartesiano. La tuple funciona igual que un struct o un registro. Porque contiene un texto Y otro texto Y un numero Y otro numero al mismo tiempo.
7. Haskell: es un tipo recursivo que además esta estructurado como una union. Se define a si mismo al incluir dos arboles binarios como hijos dentro de su propio constructor Nodo.
8. Haskell: es una Union porque los valores son excluyentes entre si. Un dato de tipo Color puede ser Rojo O Verde O Azul pero solo una opción a la vez.

4) Mutabilidad/Inmutabilidad

1. Mutabilidad: un dato es mutable cuando su contenido o estado interno puede ser modificado después de haber sido creado sin cambiar su dirección de memoria (entidad).

Inmutabilidad: un dato es inmutable cuando su estado no puede ser alterado una vez creado. Si se necesita modificar el valor, se crea un objeto nuevo en una nueva dirección de memoria para almacenar un nuevo resultado.

Ejemplos:

- Python divide estrictamente sus tipos de datos nativos entre:
 - Tipos Inmutables (Ej.: tuple, int, str):
mi_tupla = (1, 2, 3) mi_tupla[0] = 99

Si intentas modificar un elemento de un tupla, Python arrojara un error.

- Tipos mutables (Ej.: list, dict, set): `mi_lista = [1, 2, 3]` `mi_lista[0] = 99`
Las listas se pueden modificar libremente en la misma dirección de memoria
 - Ruby, la mayoría de los objetos, incluyendo String y arreglos Array, son mutables por defecto. Sin embargo, Ruby ofrece el mecanismo `.freeze` para forzar mutabilidad:
 - Comportamiento mutable por defecto:
`texto = "Hola"` `texto << " Mundo"` # modifica el objeto original ("Hola Mundo")
 - Inmutabilidad con `.freeze`: el objeto se transforma en inmutable. Cualquier intento posterior de modificarlo lanzara error en tiempo de ejecución (`FrozenError`)
`texto = "Hola".freeze` # Esto arroja un error: `FrozenError (can't modify frozen String)` `texto << " Mundo"`
2. El código presentado no demuestra mutabilidad del objeto, sino una reasignación de variable. Al hacer `new(2)`, no se está alterando el estado interno del objeto original `new(1)`, sino que se está creando un objeto nuevo en otra dirección de memoria y cambiar el puntero de la variable `a`. Para demostrar si es mutable o no, se debería intentar modificar uno de sus atributos internos y verificar si el objeto cambia su estado

5) Manejo de punteros

1. Si, el lenguaje C permite tomar el l-valor de las variables. Es una característica mas potente y fundamental del lenguaje.
Para hacerlo, C utiliza el operador de dirección `&` (ampersand):

```
#include <stdio.h>
int main() {
```

```
    int numero = 42; // 'numero' es un l-valor direc de memoria
```

```
    // Tomamos el l-valor de 'numero' usando '&'
```

```
    // y lo asignamos a la variable 'puntero'
```

```
    int *puntero = &numero; // numero = direc de memoria
```

```
    // al tener el l-valor, podemos modificar directamente esa memoria
```

```
    *puntero = 99;
```

2. Problemas en el manejo de punteros:

- Puntero Colgante: cuando liberas (eliminas) la memoria a la que apunta un puntero pero el puntero sigue guardando esa dirección de memoria vieja. Si intentas usarlo, el programa puede romperse o leer datos basura de un espacio que ya pertenece a otra variable. (por eso después de un `free(ptr) → ptr = NULL`)
- Aliasing y Efectos Colaterales: cuando dos o mas punteros apuntan exactamente a la misma dirección de memoria. Si modificas el valor a través de un puntero, cambias el resultado en cualquier otra parte del programa que use los otros punteros.

Ej.:

```
int numero = 100;
int *ptr1 = &numero;
int *ptr2 = &numero;
*ptr1 = 20;    // ahora ptr2 tambien vale 20
```

6) TAD

1. Para que una unidad sea considerada un TAD (Tipo de Dato Abstracto) debe cumplir con las siguientes características:
 - La unidad debe funcionar como una caja negra. Debe exponer un contrato publico(interfaz) que le dice al usuario que hace el TAD, pero debe ocultar como lo hace (lógica interna y algoritmos)
 - La información debe estar oculta. La unidad debe garantizar que la estructura de datos interna sea inaccesible y privada, asegurando que la única forma de interactuar con ella sea a través de las operaciones permitidas.
 - Empaquetar los dos elementos que definen cualquier tipo de dato:
 - Representación: conjunto de datos o variables que almacenan el estado (ocultos)
 - Operaciones: conjunto de funciones o procedimientos que manipulan esa representación
2. Ejemplos de TAD:
 - En Java se modelan usando Clases. La interfaz son los métodos public y la representación interna se oculta con private:

```
public interface PilaTAD {
    void apilar(int elemento);
    int desapilar();
    boolean estaVacia();
}

public class PilaImplementacion implements PilaTAD {
    private List<Integer> datos = new ArrayList<>();
```

```

    public void apilar(int elemento) {
        datos.add(elemento);
    }

```

- En ADA usa packages (paquetes) y la palabra clave private:

```

package Pila_TAD is
    type Pila is private;
    // El usuario sabe que existe, pero no qué tiene adentro

    -- OPERACIONES
    procedure Apilar(P : in out Pila; Elemento : Integer);
    procedure Desapilar(P : in out Pila; Elemento : out Integer);
    function Esta_Vacia(P : Pila) return Boolean;

private
    -- REPRESENTACIÓN: Oculta para el programa cliente
    type Arreglo_Datos is array (1 .. 100) of Integer;
    type Pila is record
        Datos : Arreglo_Datos;
        Tope : Integer := 0;
    end record;
end Pila_TAD;

```

- En C no existen las clases ni los modificadores private. Para lograr un TAD se usan puntero en el archivo cabecera (.h) ocultando la estructura real en el archivo de código (.c):

- pila.h (Interfaz):

```

// Se declara que existe la estructura, pero NO sus campos
typedef struct PilaOpaqua* PilaR;

// Operaciones públicas
PilaR crear_pila();
void apilar(PilaR p, int elemento);
int desapilar(PilaR p);

```

- pila.c (Implementación interna oculta):

```

#include "pila.h"
#include <stdlib.h>

```

```

// Acá se define la REPRESENTACIÓN. El cliente no puede ver esto.

```

```

struct PilaOpaqua {
    int datos[100];
    int tope;
};
// Implementación de las funciones
void apilar(PilaR p, int elemento) {
    p->datos[p->tope++] = elemento;
}

```

- En Python no tiene un mecanismo estricto por el compilador para bloquear el acceso a los datos, por lo que el ocultamiento de información se maneja por conversión usando un doble guion bajo para indicar que algo es privado:

```

class PilaTAD:
    def __init__(self):
        # REPRESENTACIÓN: El doble guion bajo simula el ocultamiento
        self.__datos = []

    # OPERACIONES
    def apilar(self, elemento):
        self.__datos.append(elemento)

    def desapilar(self):
        if self.esta_vacia():
            raise IndexError("La pila está vacía")
        return self.__datos.pop()

    def esta_vacia(self):
        return len(self.__datos) == 0

```